

JARVIS MASTERPLAN V5 - SYSTEMDOKUMENTATION

Stand: 12. Mai 2026

1. Vision & Ziel

Jarvis ist das lokale AI Operating System.

Es dient als zentrale Steuer-, Status- und Control-Schicht fuer digitale Aufgaben, lokale Modelle, Agenten, Voice, Memory, Trading-Analyse und spaetere Automatisierung.

Ziel:

Ein lokales System, das nicht nur reagiert, sondern Aufgaben versteht, Kontext bewertet, Agenten koordiniert, Entscheidungen vorbereitet und den Menschen als Freigabeinstanz sichtbar im Prozess haelt.

Zielbild:

Lokales AI Operating System mit Multi-Agent Runtime und Live Control Center.

2. Architekturuebersicht

Systemfluss:

USER

- > Jarvis UI / Home Dashboard
- > Hermes Router
- > Hermes Brain Status
- > Planner / Agent Dashboard
- > Spezialagenten / Statusmodule
- > Executor / Runtime Control
- > System / Tools / Dateien

Jede Ebene hat eine klare Verantwortung.

Verantwortlichkeiten:

- Jarvis: UI, Runtime, Voice, Status, Control Center
- Hermes: Brain, Router, Planner, Learning, Delegation
- Ollama: lokale Modellbereitstellung
- Agenten: Fachdomaenen und Analyse
- Executor: kontrollierte Ausfuehrung nach Freigabe
- Statusmodule: read-only Uebersicht fuer UI und Debugging

Aktuelle Kernmodule:

- Hermes Adaptive Routing
- Hermes Brain Status
- Agent Dashboard Status
- Runtime Status
- System Snapshot
- UI Status Snapshot
- Learning/Memory Status

- Voice Status
- Trading Panel Status

3. Aktueller Systemstand

Bereits vorbereitet oder implementiert:

- Hermes Router mit Adaptive Routing
- UI-freundlicher Hermes Brain Status
- Agent Dashboard mit bekannten und geplanten Agenten
- Runtime Status fuer Hermes, Ollama, Memory, Voice, Git und Runtime-Pfade
- System Snapshot als zentrale Statusaggregation
- UI Status Snapshot als JSON-Grundlage fuer Jarvis Panels
- Learning/Memory Status fuer vorhandene '.hermes/' Strukturen
- Developer/Debug Status fuer CLI-Checks und Modulverfuegbarkeit
- Voice Status als read-only Planungsstatus
- Trading Panel Status als sicherer Planungsstatus
- Trading Analyst Roadmap
- Gradio-basierte Jarvis UI mit Hermes Status Bereich

Neue oder geplante Agenten und Domaenen:

- memory_agent
- coding_agent
- research_agent
- business_agent
- improvement_agent
- trading_agent / hermes_trading_analyst

Der Trading Analyst ist geplant, sichtbar und bewusst nicht ausfuehrend.

4. Architekturregeln

- Keine direkte Ausfuehrung durch Agenten.
- Keine automatischen Commits.
- Keine automatischen Pushes.
- Keine Runtime-Daten im Git.
- Keine automatischen Trades.
- Keine Order-Funktion im Trading Analyst.
- Statusmodule sind read-only-first.
- Riskante Aktionen sind approval-gesteuert.
- Human-in-the-loop bleibt Pflicht.
- Fallbacks duerfen nicht durch UI-Statusfunktionen gebrochen werden.

Alle Ausfuehrungen laufen ueber kontrollierte Schichten. Agenten duerfen analysieren, planen und vorschlagen. Umsetzung bleibt getrennt und freigabepflichtig.

5. Mission Control

Jarvis Mission Control entwickelt sich zum Home Dashboard v1.

Das Dashboard soll den Systemzustand dauerhaft sichtbar machen, ohne Services automatisch zu starten oder Runtime-Dateien zu schreiben.

Permanente UI-Elemente:

- XAUUSD
- EURUSD

- Wetter
- aktive Agenten
- Taskline
- Hermes/Ollama Status

Zentrale Statusquellen:

- 'build_hermes_ui_status()'
- 'build_hermes_system_snapshot()'
- 'build_agent_dashboard_status()'
- 'build_runtime_status()'
- 'build_learning_memory_status()'
- 'build_voice_status()'
- 'build_trading_panel_status()'

Mission Control zeigt Status, Warnungen und naechste Aktionen. Es ersetzt keine menschliche Freigabe.

6. Multi-Agent System

Jarvis bleibt der Einstiegspunkt.

Beispiel:

User -> Jarvis -> Hermes -> Agent Dashboard -> passender Agentenkontext -> Ergebnis

Keine direkte Interaktion mit einzelnen Agenten ist notwendig.

Agent Dashboard:

- listet bekannte und geplante Agenten
- zeigt Domain, Status und Capabilities
- zeigt Safety Flags
- markiert ausfuehrbare und nicht ausfuehrbare Agenten
- macht geplante Agenten frueh in der UI sichtbar

Trading Agent:

- Status: planned
- Capabilities: market_analysis, multi_timeframe_analysis, pattern_detection, signal_alerting, prediction_feedback_learning, ctrader_integration_planned
- Safety: analysis_only, no_auto_trading, human_review_required
- can_execute: false

7. LLM Strategie

Hybrid-Ansatz:

Small Model -> schnelle Klassifikation, Routing, Statuszusammenfassung

Large Model -> komplexe Planung, Review, Architekturentscheidungen

Lokale Prioritaet:

- Ollama fuer lokale Modelle
- Privacy-Mode mit lokalem Fallback
- externe Provider nur ueber explizite Provider-Layer

Routing-Strategie:

- Hermes Adaptive Routing bestimmt Route, Intent, Domain und Agent Domain.
- Hermes Brain Status liefert UI-freundliche Zusammenfassungen.

- Provider- und Modell-Empfehlungen werden transparent ausgegeben.
- Fallback-Verhalten bleibt defensiv und stabil.

Inspirationen fuer weitere Entwicklung:

- LangGraph: zustandsorientierte Agentenfluesse
- CrewAI: Rollen, Teams und Delegation
- AutoGen: Multi-Agent-Konversationen und Review-Schleifen
- OpenDevin / SWE-Agent: Developer-Agent-Workflows
- OpenClaw: lokale Agenten-/Tool-Control-Ideen
- MCP / A2A: standardisierte Tool- und Agent-Kommunikation
- Agent Scheduler / Context Lifecycle: geplante Tasks und kontrollierter Kontext

8. Self-Improvement & Learning

Hermes lernt nicht durch versteckte Automation, sondern durch sichtbare, bewertbare Rueckmeldungen.

Aktueller Fokus:

- Learning/Memory Status read-only sichtbar machen
- Routing Hints defensiv auslesen
- Improvements sichtbar zusammenfassen
- Developer/Debug Checks zentral dokumentieren

Prediction Feedback Learning:

Hermes erstellt spaeter Prognosen:

- up
- down
- neutral

Eine Prognose speichert mindestens:

- Symbol
- Richtung
- Timeframe
- Confidence
- Setup-Kontext
- Ablaufzeit

Spaeter prueft Hermes objektiv, ob die Prognose richtig war.

Bewertungen:

- correct
- wrong
- expired
- invalidated
- late_correct

Daraus lernt Hermes:

- Pattern-Gewichtung
- Symbol-Verhalten
- Session-Verhalten
- Timeframe-Qualitaet
- Confidence-Kalibrierung

9. Sicherheit & Git

Sicherheitsprinzipien:

- human-in-the-loop
- read-only-first
- no_auto_trading
- approval-gesteuert
- keine automatischen Commits
- keine automatischen Pushes
- keine Runtime-Schreibzugriffe durch Statusmodule
- keine Secrets in Dokumentation oder Code

Trading-Sicherheit:

- Trading Analyst analysiert und alarmiert.
- Er fuehrt keine Orders aus.
- Er verwaltet keine API-Keys.
- cTrader wird zunaechst nur als QUOTE-/Chart-/Pattern-Quelle geplant.
- Order- und Broker-Funktionen bleiben ausserhalb dieses Plans.

Git bleibt menschlich kontrolliert. Codex, Agenten und Statusmodule duerfen keine Pushes ausfuehren.

10. UI Vision

Zukuenftig:

- Jarvis Home Dashboard v1
- Hermes Control Center
- Agent Dashboard Panel
- Runtime Control Panel
- Hermes Brain Panel
- Learning/Memory Panel
- Voice Panel
- Trading Panel
- Developer/Debug Panel

UI Status Snapshot:

Der zentrale Status wird als read-only JSON erzeugt und kann direkt von der UI verwendet werden.

Panels:

- chat_panel
- hermes_brain_panel
- agent_dashboard_panel
- runtime_control_panel
- learning_memory_panel
- developer_debug_panel
- voice_panel
- trading_panel

Trading Panel:

- XAUUSD, EURUSD, GER40
- HTF: W1, D1, H4
- MTF: H1, M15

- LTF: M5, M1
- Pattern: Rejection, False Break, Engulfing, Morning Star, Evening Star
- Confidence Score: 0-12
- Prediction History geplant
- Trefferquote nach Setup, Session und Timeframe geplant
- analysis_only und no_auto_trading sichtbar

Voice Panel:

- Wake Word geplant
- Whisper geplant
- Edge TTS geplant
- lokale Offline Voice geplant
- Streaming Audio geplant
- kein Mikrofonzugriff im Statusmodul

11. Roadmap

Nächste Schritte:

1. Jarvis Home Dashboard v1 anbinden
2. Hermes UI Status im Dashboard verdichten
3. Voice Runtime planen und sicher kapseln
4. cTrader QUOTE Integration designen
5. Trading Analyst als Analyseagent weiter planen
6. Prediction Feedback Loop implementieren
7. Agent Execution Sandbox haerten
8. Agent Scheduler und Context Lifecycle konzipieren
9. MCP/A2A-kompatible Schnittstellen prüfen
10. UI-Panels schrittweise mit Live-Status verbinden

Kurzfristige Tests:

```
python3 agents/core/hermes_ui_status.py
python3 agents/core/hermes_ui_status.py "Analysiere XAUUSD auf M15"
python3 agents/core/hermes_system_snapshot.py
python3 agents/core/hermes_router.py "Analysiere XAUUSD auf M15"
```

12. Klarstellung

Jarvis ist das System.

Hermes ist das Brain.

Ollama ist die lokale Modellschicht.

Agenten sind spezialisierte Arbeits- und Analysekontexte.

Trading ist Analyse und Alerting, nicht automatische Orderausführung.

Statusmodule liefern read-only Daten fuer die UI.

Das Ziel bleibt ein lokales AI Operating System mit Multi-Agent Runtime, Live Control Center und menschlicher Kontrolle ueber jede riskante Aktion.